UCS3461 Foundations of Artificial Intelligence

INFERENCE IN FIRST-ORDER LOGIC

Semester 4, B.E. Computer Science and Engineering
21 February 2026

Sakaya Milton R
SSN College of Engineering

Validity, Unsatisfiability, Entailment, Equivalence

- An interpretation I is a model of a formula A iff $I(A) = T$.
- A formula A is satisfiable iff A has a model – there is an interpretation I in which $I(A) = T$.
- A formula A is valid iff $I(A) = T$ in every interpretation I
- A formula A is unsatisfiable iff $I(A) = F$ in every interpretation I
- Formula A logically entails formula B (B is a logical consequence of A) iff $I(B) = T$ in every interpretation I in which $I(A) = T$.

$$A \models B$$

- Two formulas A and B are logically equivalent to each other iff $I(A) = I(B)$ in every interpretation suitable for A and B.

$$A \equiv B$$

Knowledge Base

- A knowledge base, KB, is a set of clauses.
- A knowledge base, KB, is true in interpretation I iff every clause in KB is true in I.
- Let g be a conjunction of atoms. g is a logical consequence of KB ($KB \models g$) iff every model of KB is a model of g.
- $KB \models g$ iff in every interpretation in which KB is true, g is true.

Ground Resolution

Propositionalization

- Inference in FOL
  - Convert the knowledge base to PL
  - Use inference in PL.
- How to convert existential quantifiers?
  Skolemize.
- How to convert universal quantifiers?
  Replace variables by ground terms.

Substitution and Instances

- Let A be a formula, x a variable, and t a term. Then

$$A[x/t]$$

  denotes an instance of A, obtained by uniformly substituting every free occurrence of x by t in A.
- Every ocurrence of the same variable is replaced by the same term.
- A substitution is a finite set of the form $[x_1/t_1, \ldots, x_n/t_n]$, where each x_i is a distinct variable and each t_i is a term.
- The application of a substitution $\sigma = [x_1/t_1, \ldots, x_n/t_n]$ to formula A is written as

$$A\sigma$$

  It is the instance of A with every occurrence of x_i replaced by t_i.

Substitution and Instances: Example

- Simultaneous substitution:

$$\{Px, Qf(y)\}[x/y, y/f(a)] = \{Py, Qf(f(a))\}$$

- Sequence of substitutions: $[x/t_1][y/t_2]$ first substitute x by t_1, and then, substitute y by t_2

$$\{Px, Qf(y)\}[x/y][y/f(a)] = \{Py, Qf(y)\}[y/f(a)] = \{Pf(a), Qf(f(a))\}$$

Ground Instances

- An instance of a formula A is obtained by substituting terms for variables.
- Substitutions which make a formula variable-free are called ground substitutions, and
- The result of applying a ground substitution to a formula is a ground instance of that formula.
- We can apply ground resolution procedure.

Gound Instances: Example

- Consider the KB:

$$\forall x\ (King(x) \land Greedy(x) \rightarrow Evil(x))$$
$$King(John)$$
$$Greedy(John)$$
$$Brother(Richard,John)$$

- Instantiate the universal formulas in all possible ways:

$$King(John) \land Greedy(John) \rightarrow Evil(John)$$
$$King(Richard) \land Greedy(Richard) \rightarrow Evil(Richard)$$
$$King(John)$$
$$Greedy(John)$$
$$Brother(Richard,John)$$

- The new KB is propositionalized: proposition symbols are King(John), Greedy(John), Evil(John), King(Richard), etc.

Ground Resolution Rule

Ground resolution rule: Let C_1 , C_2 be ground clauses such that a literal $L \in C_1$ and its complementary literal $\bar{L} \in C_2$. The clauses C_1, C_2 are said to be clashing clauses and to clash on the complementary pair of literals $L, \bar{L}$. The resolvent of C_1 and C_2, is the clause:

$$C = Res(C_1, C_2) = (C_1 - \{L\}) \cup (C_2 - \{\bar{L}\})$$

- C_1 and C_2 are the parent clauses of C.

Ground Resolution Theorem

A closed formula A in Skolem form $A = \forall y_1 \forall y_2 \ldots \forall y_k A^*$ with its matrix A^* in CNF is unsatisfiable if and only if there exists a finite sequence of clauses $C_1, C_2, \ldots, C_n$ with the properties

- For $k = 1, \ldots, n$
  - either C_k is a ground instance of some clause $C \in A^*$, i.e., C_k has the form $C_k = C[y_1/t_1][y_2/t_2] \ldots [y_k/t_k]$ where $t_1, t_2, \ldots, t_k \in D(A)$,
  - or C_k is a resolvent (in the sense of propositional logic) of two clauses C_i and C_j with $i, j < k$.
- C_n is the empty clause.

Ground Resolution: Example

$A = \forall x(p(x) \wedge \neg p(f(x)))$

Ground Resolution: Example

$A = \forall x(p(x) \wedge \neg p(f(x)))$

$A = \{ p(x), \neg p(f(x))$

$G(A) = \{ p(a), \neg p(f(a)) \},$

$\{ p(f(a)), \neg p(f(f(a))) \}, \ldots$

Ground Resolution: Example

$A = \forall x(p(x) \wedge \neg p(f(x)))$

$A = \{ p(x), \neg p(f(x)) \}$

$G(A) = \{ p(a), \neg p(f(a)) \},$

$\{ p(f(a)), \neg p(f(f(a))) \}, \ldots$

$p(a)$ $\quad \neg p(f(a))$ $\quad p(f(a))$ $\quad \neg p(f(f(a)))$

Ground Resolution: Example

$A = \forall x(p(x) \wedge \neg p(f(x)))$

$A = \{ p(x), \neg p(f(x)) \}$

$G(A) = \{ p(a), \neg p(f(a)) \},$

$\{ p(f(a)), \neg p(f(f(a))) \}, \ldots$

$p(a) \quad \neg p(f(a)) \quad p(f(a)) \quad \neg p(f(f(a)))$

$\square$

Ground Resolution: Example

$$A = \forall x \forall y((\neg p(x) \vee \neg p(f(a)) \vee q(y)) \wedge p(y) \wedge (\neg p(g(b,x)) \vee \neg q(b)))$$

$$A^* = \{\{\neg p(x), \neg p(f(a)), qy\}, \{p(y)\}, \{\neg p(g(b,x)), \neg q(b)\}\}$$

$$A^* = \{\{\neg Px, \neg Pf(a), Qy\}, \{Py\}, \{\neg Pg(b,x), \neg Qb\}\}$$

$$\begin{tikzcd}
\{\neg Px, \neg Pf(a), Qy\} \arrow[d, "{[x/f(a)]}", "{[y/b]}"'] & \{Py\} \arrow[d, "{[y/f(a)]}"'] \arrow[d, "{[y/g(b,a)]}"] & \{\neg Pg(b,x), \neg Qb\} \arrow[d, "{[x/a]}"] \\
\{\neg Pf(a), Qb\} \arrow[dr] & \{Pf(a)\} \arrow[dl] \arrow[dr] & \{Pg(b,a)\} \arrow[dl] & \{\neg Pg(b,a), \neg Qb\} \arrow[dl] \\
& \{Qb\} \arrow[dr] & & \{\neg Qb\} \arrow[dl] \\
& & \square &
\end{tikzcd}$$

Propositionalization: Properties

- The new KB entails a ground formula iff the original KB entails the ground formula.
- Every FOL KB can be propositionalized so as to preserve entailment.
- Propositionalize KB and query, apply resolution, return answers.

Propositionalization: Problems

- Propositionalization generates a lot of irrelevant sentences.
  - Consider the KB:

    $\forall$x King(x) $\wedge$ Greedy(x) $\rightarrow$ Evil(x)
    King(John)
    $\forall$y Greedy(y)
    Brother(Richard,John)
  - It seems obvious that Evil(John), but propositionalization produces a lot of facts such as Greedy(Richard) that are irrelevant.
- With function symbols, there are infinitely many ground terms.

  Father(John)
  Father(Father(John))
  Father(Father(Father(John)))
- With p k-ary predicates and n constants, there are $p \times n^k$ instantiations!

Unification

Substitution and Instances

- Let A be a formula, x a variable, and t a term. Then

$$A[x/t]$$

  denotes an instance of A, obtained by uniformly substituting t for every free occurrence of x in A.
- Every ocurrence of the same variable is replaced by the same term.
- A substitution is a finite set of the form $[x_1/t_1, \ldots, x_n/t_n]$, where each x_i is a distinct variable and each t_i is a term.
- The application of a substitution $\sigma = [x_1/t_1, \ldots, x_n/t_n]$ to formula A is written as

$$A\sigma$$

  It is the instance of A with every occurrence of x_i replaced by t_i.

Substitution and Instances: Example

- Simultaneous substitution:

$$\{Px, Qf(y)\}[x/y, y/f(a)] = \{Py, Qf(f(a))\}$$

- Sequence of substitutions: $[x/t_1][y/t_2]$ first substitute x by t_1, and then, substitute y by t_2

$$\{Px, Qf(y)\}[x/y][y/f(a)] = \{Py, Qf(y)\}[y/f(a)] = \{Pf(a), Qf(f(a))\}$$

Substitution Application: Examples

- Consider these substitutions:

$$\sigma_1 = [x/a, y/b, z/c, d/e]$$
$$\sigma_2 = [a/x, y/b, c/z, d/e]$$
$$\sigma_3 = [a/v, x/v, y/b, c/w, z/w, d/e]$$

- Apply substitutions:

$$\begin{aligned}
(Pabcd)\sigma_1 &= (Pabcd)[x/a, y/b, z/c, d/e] &&= Pabce \\
(Pxyze)\sigma_1 &= (Pxyze)[x/a, y/b, z/c, d/e] &&= Pabce \\
(Pabcd)\sigma_2 &= (Pabcd)[a/x, y/b, c/z, d/e] &&= Pxbze \\
(Pxyze)\sigma_2 &= (Pxyze)[a/x, y/b, c/z, d/e] &&= Pxbze \\
(Pabcd)\sigma_3 &= (Pabcd)[a/v, x/v, y/b, c/w, z/w, d/e] &&= Pvbwe \\
(Pxyze)\sigma_3 &= (Pxyze)[a/v, x/v, y/b, c/w, z/w, d/e] &&= Pvbwe
\end{aligned}$$

Unifier

- An expression is a term, an atom, a clause, or a set of clauses.
- Let E be an expression and let $\sigma = [x_1/t_1, \ldots, x_n/t_n]$ be a substitution. An instance $E\sigma$ of E is obtained by simultaneously replacing each occurrence of x_i in E by t_i.
- Substitution σ is a unifier of E_1 and E_2 (atoms or terms) if $E_1\sigma = E_2\sigma$. E_1 and E_2 are unifiable.
- Substitution σ is a most general unifier (mgu) of E_1 and E_2 if
  - σ is a unifier of E_1 and E_2 and
  - if substitution σ' also unifies E_1 and E_2, then $E\sigma'$ is an instance of $E\sigma$ for all atoms E.

$$E\sigma' = E\sigma\theta$$

Unifier: Examples

$Pf(x)g(y), \neg Pf(f(a))g(z)$

- $\sigma = [x/f(a), z/y]$
  $Pf(f(a))g(y), \neg Pf(f(a))g(y)$
- $\sigma' = [x/f(a), y/f(g(a)), z/f(g(a))]$
  $Pf(f(a))g(f(g(a))), \neg Pf(f(a))g(f(g(a)))$

$$\begin{aligned}\sigma' &= [x/f(a), y/f(g(a)), z/f(g(a))] \\ &= [x/f(a), z/y][y/f(g(a))] \\ &= \sigma[y/f(g(a))]\end{aligned}$$

- $\sigma'' = [x/f(a), y/a, z/a]$
  $Pf(f(a))g(a), \neg Pf(f(a))g(a)$

$$\begin{aligned}\sigma'' &= [x/f(a), y/a, z/a] \\ &= [x/f(a), z/y][y/a] \\ &= \sigma[y/a]\end{aligned}$$

Unification: Example

1. $A = \{Pf(z, g(a, y))h(z), Pf(f(u, v), w)h(f(a, b)))\}$
2. $Pf(z, g(a, y))h(z)[z/f(u, v)]$
   $Pf(f(u, v), w)h(f(a, b))$
3. $Pf(f(u, v), g(a, y)), h(f(u, v))$
   $Pf(f(u, v), w), h(f(a, b))[w/g(a, y)]$
4. $Pf(f(u, v), g(a, y)), h(f(u, v))[u/a]$
   $Pf(f(u, v), g(a, y)), h(f(a, b))$
5. $Pf(f(a, v), g(a, y)), h(f(a, v))[v/b]$
   $Pf(f(a, v), g(a, y)), h(f(a, b))$
6. $Pf(f(a, v), g(a, y)), h(f(a, b))$
   $Pf(f(a, v), g(a, y)), h(f(a, b))$

Unifier: $[z/f(u, v)][w/g(a, y)][u/a][v/b]$

Unification

Algorithm: Unify(t_1, t_2)

Input: t_1, t_2: terms (functions, constants, variables)
Output: most general unifier of t_1 and t_2 if it exists or Failure otherwise
Local: E: a set of equality statements
S: substitution

Algorithm: Unify (t_1, t_2)

```
E := {t1 = t2}// Set of equality statements
S := {} // Substitution
while E ≠ {} do
    select and remove x = y from E
    if x is not identical to y then
        if x is a variable then
            replace x with y in E and S
            S := S ∪ {x/y}
        else if y is a variable then
            replace y with x in E and S
            S := S ∪ {y/x}
        else if x is f(x1, ..., xn) and y is f(y1, ..., yn) then
            E := E ∪ {x1 = y1, ..., xn = yn}
        else
            return F // t1 and t2 do not unify
return S // S is mgu of t1 and t2
```

Algorithm: Unify (t_1, t_2)

```
E := {t1 = t2}// Set of equality statements
S := {} // Substitution
while E ≠ {} do
    select and remove x = y from E
    if x is not a variable then  swap x and y
    if x is not identical to y then
        if x is a variable then
            replace x with y in E and S
            S := S ∪ {x/y}
        else if x is f(x1, ..., xn) and y is f(y1, ..., yn) then
            E := E ∪ {x1 = y1, ..., xn = yn}
        else
            return F // t1 and t2 do not unify
return S // S is mgu of t1 and t2
```

Algorithm: Unify (t_1, t_2)

```
E := {t1 = t2}// Set of equality statements
S := {} // Substitution
while E ≠ {} do
    select and remove x = y from E
    if x is not a variable then  swap x and y
    if x is not identical to y then
        if x is a variable then
            if x occurs in y then
                return F // t1 and t2 do not unify
            replace x with y in E and S
            S := S ∪ {x/y}
        else if x is f(x1, ..., xn) and y is f(y1, ..., yn) then
            E := E ∪ {x1 = y1, ..., xn = yn}
        else
            return F // t1 and t2 do not unify
return S // S is mgu of t1 and t2
```

Unification: Example

E	S
$P(f(z, g(a, y)), h(z))$ $= P(f(f(u, v), w), h(f(a, b)))$	$[\ ]$
$f(z, g(a, y)) = f(f(u, v), w)$ $h(z) = h(f(a, b))$	$[\ ]$
$z = f(u, v), g(a, y) = w, h(z) = h(f(a, b))$	$[\ ]$
$g(a, y) = w, h(f(u, v)) = h(f(a, b))$	$[z/f(u, v)]$
$h(f(u, v)) = h(f(a, b))$	$[z/f(u, v), w/g(a, y)]$
$f(u, v) = f(a, b)$	$[z/f(u, v), w/g(a, y)]$
$u = a, v = b$	$[z/f(u, v), w/g(a, y)]$
$v = b$	$[z/f(a, v), w/g(a, y), u/a]$
	$[z/f(a, b), w/g(a, y), u/a, v/b]$

General Resolution Rule

- General resolution rule: Let C_1 , C_2 be clauses with no variables in common. Let $L_1 \in C_1$ and $L_2 \in C_2$ be literals such that L_1 and $\overline{L_2}$ are unifiable by an mgu σ ($L_1\sigma = \overline{L_2}\sigma$). C_1, C_2 are said to be clashing clauses and to clash on the literals L_1, L_2. The resolvent of C_1 and C_2, is the clause:

$$C = Res(C_1, C_2) = (C_1\sigma - \{L_1\sigma\}) \cup (C_2\sigma - \{L_2\sigma\})$$

- Binary resolution rule
- Factoring: When taking the union of the clauses in the resolution rule, collapse identical literals.
- No names in common: standardized apart.

General Resolution Procedure

Algorithm: Resolution(K, G)

Input: K, a set of clauses
Output: $K \cup \{G\}$ is satisfiable or unsatisfiable

$C := K$
loop
    Choose a new pair of clashing clauses $\{C_1, C_2\} \subset C$
    if no new pair exists in C **then**
        return satisfiable
    $R := Res(C_1, C_2)$
    if $R = \square$ **then**
        return unsatisfiable
    $C := C \cup \{R\}$

Lifiting Lemma

- Let $C_1^{'}, C_2^{'}$ be ground instances of C_1, C_2, respectively. Let C' be a ground resolvent of $C_1^{'}$ and $C_2^{'}$. Then there is a resolvent C of C_1 and C_2 such that C' is a ground instance of C.

$$\begin{tikzcd}
C_1, C_2 \arrow[r, "\text{Resolution}"] \arrow[d, "\text{Instance}"'] & C \arrow[d, "\text{Instance}"] \\
C_1^{'}, C_2^{'} \arrow[r, "\text{Ground Resolution}"'] & C'
\end{tikzcd}$$

General Resolution: Example

1 $\{\neg Px, Qx, Rxf(x)\}$

2 $\{\neg Px, Qx, Sf(x)\}$

3 $\{Ta\}$

4 $\{Pa\}$

5 $\{\neg Ray, Ty\}$

6 $\{\neg Tx, \neg Qx\}$

7 $\{\neg Tx, \neg Sx\}$

8 $\{\neg Qa\}$ 3, 6, x/a

9 $\{Qa, Sf(a)\}$ 2, 4, x/a

10 $\{Sf(a)\}$ 8, 9

11 $\{Qa, Raf(a)\}$ 1, 4, x/a

12 $\{Raf(a)\}$ 8, 11

13 $\{Tf(a)\}$ 5, 12, $y/f(a)$

14 $\{\neg Sf(a)\}$ 7, 13, $x/f(a)$

15 $\{\square\}$ 10, 14

Top-Down Proof Procedure

Top-Down Proof Procedure in PL

- An answer clause is of the form

$$a_1 \wedge a_2 \wedge \ldots \wedge a_m \rightarrow yes$$

  Select an atom, say a_1, in its body — subgoal.
- Choose a clause in KB with a_1 as the head (if there is no such clause, fail).

$$b_1 \wedge b_2 \wedge \ldots \wedge b_p \rightarrow a_1$$

- Resolve them into

$$b_1 \wedge b_2 \wedge \ldots \wedge b_p \wedge a_2 \wedge \ldots a_m \rightarrow yes$$

- An answer is an answer clause with an empty body: $\rightarrow yes$

Top-Down Proof Procedure in PL

Algorithm: Prove_Definite_Clause_TD(KB, Query):non-deterministic

Input: KB: a set of definite clauses
Query: $Q_1 \wedge \ldots Q_k$
Output: yes if $KB \models$ Query; fails otherwise
Local: G is an answer clause / goal set

$G := Q_1 \wedge \ldots Q_k \rightarrow Y$
repeat
select an atom a_1 from the body of G
choose a clause $B \rightarrow a_1$ in KB with a_1 as head `// non-deterministic`
replace a_1 in the body of G by B
until G is an answer ($G = \rightarrow$ yes)

Top-Down Proof Procedure in FOL

- A generalized answer clause is of the form

$$a_1 \wedge a_2 \wedge \ldots \wedge a_m \rightarrow Y(t_1, \ldots, t_k)$$

  where $t_1, \ldots, t_k$ are terms and $a_1, \ldots, a_m$ are atoms.
- Select atom in the body to resolve against, say a_1.
- The SLD resolution of this generalized answer clause on a_1 with the clause

$$b_1 \wedge b_2 \wedge \ldots \wedge b_p \rightarrow a_1'$$

  where a_1 and a_1' have most general unifier σ, is

$$b_1 \wedge b_2 \wedge \ldots \wedge b_p \wedge a_2 \wedge \ldots a_m)\sigma \rightarrow (Y(t_1, \ldots, t_k)$$

Binary Resolution Rule for Definite Clauses

$$\frac{a_1 \wedge a_2 \wedge \ldots \wedge a_m \rightarrow Y(t_1, \ldots, t_k) \quad b_1 \wedge b_2 \wedge \ldots \wedge b_p \rightarrow a_1'}{b_1 \wedge b_2 \wedge \ldots \wedge b_p \wedge a_2 \wedge \ldots a_m)\sigma \rightarrow (Y(t_1, \ldots, t_k)}$$

where a_1 and a_1' have the most general unifier σ $(a_1\sigma = a_1'\sigma)$

Top-Down Proof Procedure in FOL

Algorithm: Prove_DC_TD(KB, Q): non-deterministic

Input: KB: a set of definite clauses
Query Q: a set of atoms to prove, with variables $v_1, \ldots, v_k$
Output: substitution σ if $KB \models Q\sigma$ and fail otherwise
Local: G is a generalized answer clause

$G :=$ generalized answer clause $Q \rightarrow Y(v_1, \ldots, v_k)$
while G is not an answer **do**
Suppose G is $a_1 \wedge a_2 \wedge \ldots \wedge a_m \rightarrow Y(t_1, \ldots, t_k)$
select atom a_1 in the body of G
choose clause $b_1 \wedge \ldots b_p \rightarrow a_1'$ in KB
Rename all variables in $b_1 \wedge \ldots b_p \rightarrow a_1'$ to have new names
Let σ be Unify(a_1, a_1'). Fail if Unify returns False
$G := (b_1 \wedge \ldots \wedge b_p \wedge a_2 \wedge \ldots \wedge a_m \rightarrow Y(t_1, \ldots, t_k))\sigma$
return $\{v_1 = t_1, \ldots, v_k = t_k\}$ where G is $\rightarrow Y(t_1, \ldots, t_k)$

Example

1	$\mathsf{Connected}(y, z) \wedge \mathsf{Live}(z) \rightarrow \mathsf{Live}(y).$	
2	$\rightarrow \mathsf{Live}(\mathsf{outside}).$	
3	$\rightarrow \mathsf{Connected}(w_6, w_5).$	
4	$\rightarrow \mathsf{Connected}(w_5, \mathsf{outside}).$	
5	$\mathsf{Live}(v) \rightarrow .$	
6	$\mathsf{Live}(v) \rightarrow \mathsf{Yes}(v).$	
7	$\mathsf{Connected}(v, z_1) \wedge \mathsf{Live}(z_1) \rightarrow \mathsf{Yes}(v).$	$[y/v], 1, 6$
8	$\mathsf{Live}(w_5) \rightarrow \mathsf{Yes}(w_6).$	$[v/w_6, z_1/w_5], 3, 7$
9	$\mathsf{Connected}(w_5, z_2) \wedge \mathsf{Live}(z_2) \rightarrow \mathsf{Yes}(w_6).$	$[y/w_5], 1, 8$
10	$\mathsf{Live}(\mathsf{outside}) \rightarrow \mathsf{Yes}(w_6).$	$[z_2/\mathsf{outside}], 4, 9$
11	$\rightarrow \mathsf{Yes}(w_6).$	$2, 10$

Bottom-Up Proof Procedure

Modus Ponens

- Inference rule: $\rightarrow$ elimination (Modus Ponens)

$$\frac{b, \quad b \rightarrow h}{h}$$

- Modus Ponens

$$\frac{b_1, \quad b_2, \quad \ldots \quad b_m, \quad b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow h}{h}$$

- If $b_1 \wedge \ldots \wedge b_m \rightarrow h$ is a definite clause in the knowledge base, and each b_i has been derived, then h can be derived.

Bottom-Up Proof Procedure

Algorithm: Prove_DC_BU(KB)

Input: KB, a set of definite clauses
Output: Set of all atoms that are logical consequences of KB
Local: C is a set of consequence atoms, consequence set

$C := \{\}$
repeat
    select a clause $b_1 \wedge \ldots \wedge b_m \rightarrow h$ in KB
        where $b_1, b_2, \ldots, b_m \in C$,
        and $h \notin C$
    $C := C \cup \{h\}$
until no more clauses can be selected
return C

- When the proof procedure terminates, if $g \in C$, then $KB \vdash g$.

Generalized Modus Ponens (GMP)

- Generalized Modus Ponens

$$\frac{b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow h, \quad b'_1, b'_2, \ldots, b'_m \text{ where } b'_i\theta = b_i\theta \text{ for all } i}{h\theta}$$

- GMP used with KB of definite clauses.
- All variables assumed universally quantified.
- GMP can be used in bottom-up proof procedure (forward chaining)

Generalized Modus Ponens

- Generalized Modus Ponens

$$\frac{b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow h, \quad b'_1, b'_2, \ldots, b'_m \text{ where } b'_i\theta = b_i\theta \text{ for all } i}{h\theta}$$

$$\frac{\text{King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x), \text{King(John)}, \text{Greedy}(y)}{\text{Evil}(x)}$$

$$b_1 = \text{King}(x) \qquad b'_1 = \text{King(John)}$$

$$b_2 = \text{Greedy}(x) \qquad b'_2 = \text{Greedy}(y)$$

$$h = \text{Evil}(x) \qquad h' = \text{Evil(John)}$$

$$\frac{\text{King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x), \text{King(John)}, \text{Greedy}(y)\{x/\text{John}, y/\text{John}\}}{\text{Evil}(x)\{x/\text{John}, y/\text{John}\}}$$

$$= \frac{\text{King(John)} \wedge \text{Greedy(John)} \rightarrow \text{Evil}(x), \text{King(John)}, \text{Greedy(John)}}{\text{Evil(John)}}$$

Bottom-Up Proof Procedure in FOL

Algorithm: Prove_DC_BU(KB)

Input: KB, a set of definite clauses
Output: Set of all atoms that are logical consequences of KB
Local: C is a set of consequence atoms, consequence set

$C := \{\}$
repeat
    select clause $b_1 \wedge b_2 \ldots \wedge b_m \rightarrow h$ in KB such that
        there exist atoms $b'_1, b'_2, \ldots, b'_m \in C$
        and b_i and b'_i are unified by an mgu θ for all i ($b_i\theta = b'_i\theta$ for all i)
        and there is no $h' \in C$ and θ' such that $h\theta = h'\theta'$
    $C := C \cup \{h\theta\}$
until no more clauses can be selected.

- $KB \vdash g$ if there is g' added to C in this procedure where $g = g'\theta$.

Bottom-Up Proof: Example

1	$\text{Connected}(y, z) \wedge \text{Live}(z) \rightarrow \text{Live}(y)$.	
2	$\text{Live}(\text{outside})$.	
3	$\text{Connected}(w_6, w_5)$.	
4	$\text{Connected}(w_5, \text{outside})$.	
5	$\text{Live}(\text{outside})$.	
6	$\text{Connected}(w_6, w_5)$.	
7	$\text{Connected}(w_5, \text{outside})$.	
8	$\text{Live}(w_5)$	$1, 5, 7, [y/w_5, z/\text{outside}]$
9	$\text{Live}(w_6)$	$1, 6, 8, [y/w_6, z/w_5]$

Bottom-Up Proof is Sound

Every atom in C is a logical consequence of KB (If $KB \vdash g$ then $KB \models g$).

1	Suppose there is a g that $KB \vdash g$ but $KB \not\models g$.	
2	Then there must be a first atom added to C that has an instance that is not true in every model of KB. Call it h'.	
3	Let I be a model of KB in which h' is false.	1
4	There must be an instance of clause in KB of the form $h \leftarrow b_1 \wedge \ldots \wedge b_m$ where $h' = h\theta$, and $b'_i\theta = b_i\theta \in C$ for all i.	
5	Each $b_i\theta$ is true in I.	2, 4
6	h is false in I.	
7	Therefore, an instance of this clause is false in I.	
8	Therefore, I is not a model of KB.	
9	Contradiction	

Bottom-Up Proof Fixed Point

- Output C is a fixed point – no more atoms could be added to C by the inference rule application.
- Herbrand interpretation: The domain is the set of constants. Invent a constant if the KB or query doesn't contain one.
- Let I be the Herbrand interpretation in which every ground instance of every atom in the fixed point is true and every other atom is false.
- I is a model of KB.
  Suppose $h \leftarrow b_1 \wedge b_2 \ldots b_m \in KB$ is false in I.
- $b_1, \ldots, b_m$ are in the fixed point, and h is not in the fixed point.
- By construction, the inference rule can be used to add h to the fixed point, a contradiction to it being the fixed point.
- I is the minimal model in the sense that it has the fewest true propositions. Every other model must also assign the atoms in C to be true.

Bottom-Up Proof is Complete for Datalog

- If $KB \models g$ then $KB \vdash g$.
- Suppose $KB \models g$. Then g is true in all models of KB.
- Thus g is true in the minimal model.
- Thus g is in the fixed point.
- Thus g is generated by the bottom up algorithm.
- Thus $KB \vdash g$.
- In Datalog, bottom-up procedure always halts.
- With function symbols, it may go on indefitely.

Top-Down Proof by Resolution

Top-Down Proof: Example

1	$\mathsf{Connected}(y, z) \wedge \mathsf{Live}(z) \rightarrow \mathsf{Live}(y).$	
2	$\rightarrow \mathsf{Live}(\mathsf{outside}).$	
3	$\rightarrow \mathsf{Connected}(w_6, w_5).$	
4	$\rightarrow \mathsf{Connected}(w_5, \mathsf{outside}).$	
5	$\mathsf{Live}(v) \rightarrow .$	
6	$\mathsf{Connected}(v, z_1) \wedge \mathsf{Live}(z_1) \rightarrow .$	$[y/v], 1, 5$
7	$\mathsf{Live}(w_5) \rightarrow .$	$[v/w_6, z_1/w_5], 3, 6$
8	$\mathsf{Connected}(w_5, z_2) \wedge \mathsf{Live}(z_2) \rightarrow .$	$[y/w_5], 1, 7$
9	$\mathsf{Live}(\mathsf{outside}) \rightarrow .$	$[z_2/\mathsf{outside}], 4, 8$
10	$\rightarrow .$	$2, 9$

Top-Down Proof: Exercise

1 $\rightarrow \mathsf{Upon}(a, b)$
2 $\rightarrow \mathsf{Upon}(b, c)$
3 $\rightarrow \mathsf{Blue}(a)$
4 $\mathsf{Upon}(x, y) \wedge \mathsf{Blue}(x) \rightarrow \mathsf{Blue}(y)$.
5 $\mathsf{Blue}(c) \rightarrow$

Resolution Refutation: Exercise

1 $\forall x, y.(\mathsf{Package}(x) \wedge \mathsf{Package}(y) \wedge \mathsf{Inroom}(x, 27) \wedge \mathsf{Inroom}(y, 28) \rightarrow \mathsf{Smaller}(x, y$
2 $\mathsf{Package}(a)$
3 $\mathsf{Package}(b)$
4 $\mathsf{Inroom}(a, 27) \vee \mathsf{Inroom}(a, 28)$
5 $\mathsf{Inroom}(b, 27)$
6 $\neg\mathsf{Smaller}(b, a)$
7 $\mathsf{Inroom}(a, 27) \rightarrow$

1 $\mathsf{P}x \wedge \mathsf{P}y \wedge \mathsf{I}x27 \wedge \mathsf{I}y28 \rightarrow \mathsf{S}xy$
2 $\mathsf{P}a$
3 $\mathsf{P}b$
4 $\mathsf{I}a27 \vee \mathsf{I}a28$
5 $\mathsf{I}b27$
6 $\neg\mathsf{S}ba$
7 $\mathsf{I}a27 \rightarrow$

Knowledge Base: Example

1. It is a crime for an American to sell weapons to hostile nations:
   $\forall x \forall y \forall z$ (American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\rightarrow$ Criminal(x))
2. Nono ... has some missiles.
   $\exists x$ (Owns(Nono,x) $\wedge$ Missile(x))
   Owns(Nono,M_1)
3. Missile(M_1)
4. All of its missiles were sold to it by Colonel West.
   $\forall x$ (Missile(x) $\wedge$ Owns(Nono,x) $\rightarrow$ Sells(West,x,Nono))
5. Missiles are weapons.
   $\forall x$ (Missile(x) $\rightarrow$ Weapon(x))
6. An enemy of America counts as "hostile".
   $\forall x$ (Enemy(x,America) $\rightarrow$ Hostile(x))
7. Best, who is American ...
   American(Best)
8. The country Nono, an enemy of America.
   Enemy(Nono,America)

Bottom-Up Proof: Example

1	$Cx \leftarrow Ax \wedge Wy \wedge Sxyz \wedge Hz$	
2	Onm_1	
3	Mm_1	
4	$Sbxn \leftarrow Mx \wedge Onx$	
5	$Wx \leftarrow Mx$	
6	$Hx \leftarrow Exa$	
7	Ab	
8	Ena	
9	Hn	$6, 8, [x/n]$
10	Wm_1	$3, 5, [x/m_1]$
11	Sbm_1n	$2, 3, 4, [x/m_1]$
12	Cb	$1, 7, 10, 11, 9, [x/b, y/m_1]$

Top-down Proof: Example

1	$Cx_1 \leftarrow Ax_1 \wedge Wy \wedge Sx_1yz \wedge Hz$	
2	Onm_1	
3	Mm_1	
4	$Sbx_2n \leftarrow Mx_2 \wedge Onx_2$	
5	$Wx_3 \leftarrow Mx_3$	
6	$Hx_4 \leftarrow Ex_4a$	
7	Ab	
8	Ena	
9	$\leftarrow Cb$	goal
10	$\leftarrow Ab \wedge Wy \wedge Sbyz \wedge Hz$	$1, [x_1/b]$
11	$\leftarrow Wy \wedge Sbyz \wedge Hz$	7
12	$\leftarrow My \wedge Sbyz \wedge Hz$	$5, [x_3/y]$
13	$\leftarrow Sbm_1z \wedge Hz$	$3, [y/m_1]$
14	$\leftarrow Mm_1 \wedge Onm_1 \wedge Hz$	$4, [x_2/m_1]$
15	$\leftarrow Onm_1 \wedge Hz$	3
16	$\leftarrow Hz$	2
17	$\leftarrow Eza$	$6, [x_4/z]$
18	$\leftarrow$	$8, [z/n]$

$Cx_1 \leftarrow Ax_1 \wedge Wy \wedge Hz \wedge Sxyz \qquad \leftarrow Cb$

$Ab \qquad \leftarrow Ab \wedge Wy \wedge Hz \wedge Sbyz$

$Wx_3 \leftarrow Mx_3 \qquad \leftarrow Wy \wedge Hz \wedge Sbyz$

$Mm_1 \qquad \leftarrow My \wedge Hz \wedge Sbyz$

$Hx_4 \leftarrow Ex_4a \qquad \leftarrow Hz \wedge Sbm_1z$

$Ena \qquad \leftarrow Eza \wedge Sbm_1z$

$Sbx_2n \leftarrow Mx_2 \wedge Onx_2 \leftarrow Sbm_1n$

$Mm_1 \qquad \leftarrow Mm_1 \wedge Onm_1$

$Onm_1 \qquad \leftarrow Onm_1$

$\leftarrow$

Queries and Answers

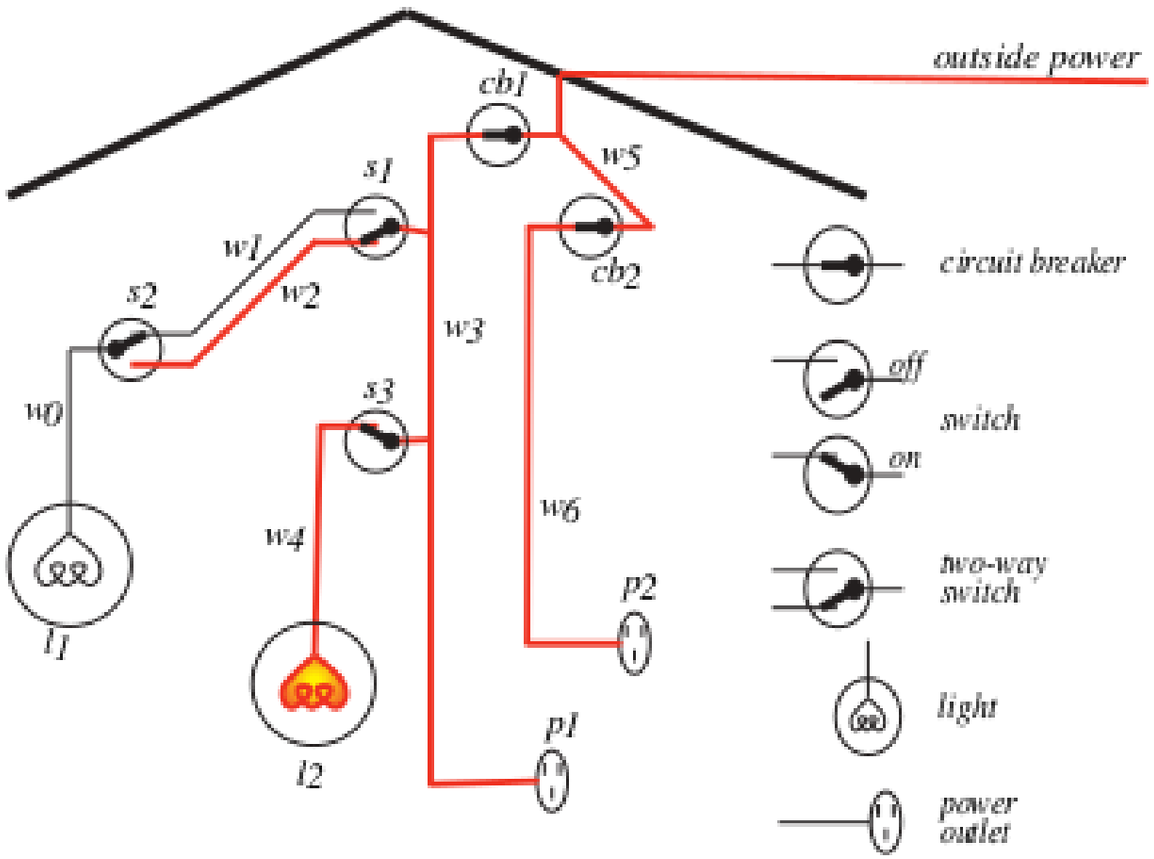
Queries and Answers

- A query is a way to ask if a body is a logical consequence of the knowledge base:

$$? \; b_1 \wedge \ldots \wedge b_m.$$

- An answer is
  - either an instance of the query that is a logical consequence of the knowledge base KB,
  - or no, if no instance of the query is a logical consequence of KB.

Example: Electrical Environment



$light(l_1).$

$light(l_2).$

$down(s_1).$

$up(s_2).$

$up(s_3).$

$ok(l_1).$

$ok(l_2).$

$ok(cb_1).$

$ok(cb_2).$

$? \; light(l_1). \quad \Rightarrow yes$

$? \; light(l_6). \quad \Rightarrow no$

$? \; up(X). \quad \Rightarrow up(s_2), up(s_3)$

Example: Electrical Environment

$connected_to(w_0, w_1) \leftarrow up(s_2).$

$connected_to(w_0, w_2) \leftarrow down(s_2).$

$connected_to(w_1, w_3) \leftarrow up(s_1).$

$connected_to(w_2, w_3) \leftarrow down(s_1).$

$connected_to(w_4, w_3) \leftarrow up(s_3).$

$connected_to(p_1, w_3).$

$lit(L) \leftarrow light(L) \wedge ok(L) \wedge live(L).$

$live(Y) \leftarrow connected_to(Y, Z) \wedge live(Z).$

$live(outside)$

$?\ connected_to(w_0, W). \quad \Rightarrow W = w_1$

$?\ connected_to(w_1, W). \quad \Rightarrow no$

$?\ connected_to(Y, w_3) \quad \Rightarrow Y = w_2, Y = w_4, Y = p_1.$

$?\ connected_to(X, W). \quad \Rightarrow X = w_0, W = w_1, \ldots$

End